

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN

Nguyễn Đăng Nam Kha

Đăng Nguyễn Huy Hoàng

BÁO CÁO MÔN HỌC
LUỒNG XỬ LÝ DỮ LIỆU THỜI GIAN THỰC CHO VIỆC
DỰ ĐOÁN SỰ PHỔ BIẾN CỦA NỘI DUNG NHẪM TỐI
UÙ BỘ NHỚ ĐỆM TẠI BIÊN

**A Real-Time Data Pipeline for Content Popularity Prediction to
Optimize Edge Caching**

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN

Nguyễn Đăng Nam Kha – 23520665
Đặng Nguyễn Huy Hoàng - 23520512

BÁO CÁO MÔN HỌC
LUỒNG XỬ LÝ DỮ LIỆU THỜI GIAN THỰC CHO VIỆC
DỰ ĐOÁN SỰ PHỔ BIẾN CỦA NỘI DUNG NHẪM TỐI
ƯU BỘ NHỚ ĐỆM TẠI BIÊN

**A Real-Time Data Pipeline for Content Popularity Prediction to
Optimize Edge Caching**

GIẢNG VIÊN HƯỚNG DẪN
Th.S Nguyễn Hồ Duy Trí

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy Nguyễn Hồ Duy Trí – giảng viên hướng dẫn trực tiếp của đề tài. Thầy đã định hướng nghiên cứu, cũng như đưa ra những góp ý chuyên môn vô cùng thiết thực trong quá trình thiết kế kiến trúc hệ thống, xử lý luồng dữ liệu lớn và tinh chỉnh mô hình máy học. Nhờ sự hỗ trợ của thầy, chúng em mới có thể hoàn thiện tốt báo cáo môn học này.

Mặc dù đã nỗ lực hết sức trong quá trình nghiên cứu và thực hiện đề tài "Luồng xử lý dữ liệu thời gian thực cho việc dự đoán sự phổ biến của nội dung nhằm tối ưu bộ nhớ đệm tại biên", nhưng do những giới hạn về mặt thời gian, nguồn lực phân cứng cũng như kinh nghiệm thực tiễn, báo cáo chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự lượng thứ cùng những lời góp ý, chỉ bảo quý báu từ thầy để đề tài có thể hoàn thiện hơn, tạo tiền đề vững chắc cho các hướng phát triển trong tương lai. Cuối cùng, chúng em kính chúc thầy luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công trong sự nghiệp giảng dạy và nghiên cứu.

Trân trọng,

Nhóm sinh viên thực hiện:

Nguyễn Đăng Nam Kha – Đặng Nguyễn Huy Hoàng

MỤC LỤC

Chương 1. BỘ DỮ LIỆU VÀ XỬ LÝ ĐẶC TRƯNG.....	3
1.1. Bối cảnh nghiên cứu.....	3
1.2. Bài báo nền tảng: Ripple.....	3
1.3. Lý do chọn đề tài.....	4
1.4. Mục tiêu của đề tài	5
1.5. Phạm vi thực hiện.....	5
Chương 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	7
2.1. Edge caching và data deduplication.....	7
2.2. Apache Kafka.....	7
2.3. Apache Spark và Structured Streaming	7
2.4. Redis.....	8
2.5. Delta Lake	8
2.6. XGBoost.....	8
2.7. Tổng hợp vai trò của các công nghệ	8
Chương 3. BỘ DỮ LIỆU VÀ XỬ LÝ ĐẶC TRƯNG.....	10
3.1. Tổng quan về bộ dữ liệu.....	10
3.2. Xử lý đặc trưng	10
Chương 4. THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG	12
4.1. Kiến trúc tổng thể hệ thống.....	12
4.2. Mở rộng mã nguồn Ripple để giả lập môi trường phân tán.....	14
4.3. Cấu hình Kafka topic	15
4.4. Luồng Realtime	15
4.5. Luồng Batch.....	16

4.6. Tích hợp với Ripple	16
Chương 5. HUẤN LUYỆN MÔ HÌNH BAN ĐẦU VÀ THỰC NGHIỆM	18
5.1. Thiết lập huấn luyện.....	18
5.2. Đánh giá mô hình máy học	19
5.3. Cấu hình vận hành thí nghiệm	21
5.4. Các độ đo đánh giá hệ thống.....	21
5.5. Kết quả thực nghiệm và phân tích kết quả.....	22
Chương 6. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN	24
6.1. Hạn chế hiện tại.....	24
6.2. Hướng phát triển	24
6.3. Kết luận	25

DANH MỤC HÌNH

Hình 1. Phân bổ số lượng Request.....	4
Hình 2. Sơ đồ mô hình hệ thống tổng thể	13
Hình 3. Sơ đồ luồng Batch	14
Hình 4. Báo cáo phân loại của mô hình trên Part001	18
Hình 5. Báo cáo phân loại của mô hình trên Part003	19
Hình 6. Báo cáo phân loại của mô hình trên Part001	20
Hình 7. Ma trận nhầm lẫn của mô hình trênn Part001 với Threshold 0.95	20
Hình 8. Biểu đồ Độ quan trọng của các đặc trưng.....	21

DANH MỤC BẢNG

Bảng 1. Tổng hợp vai trò của các công nghệ	8
Bảng 2. Các độ đo đánh giá	22
Bảng 3. Bảng kết quả so sánh trước và sau khi tích hợp ML	22

TÓM TẮT BÁO CÁO

Báo cáo trình bày giải pháp xây dựng luồng xử lý dữ liệu thời gian thực nhằm dự đoán sự phổ biến của nội dung, từ đó tối ưu hóa bộ nhớ đệm tại máy chủ biên (Edge Caching). Đề tài lấy cảm hứng từ thuật toán xóa trùng lặp dữ liệu phi tập trung Ripple. Tuy nhiên, do Ripple nguyên bản chỉ vận hành thụ động khi bộ nhớ đầy và thiếu khả năng nhận thức giá trị tương lai của dữ liệu, nhóm nghiên cứu đã đề xuất tích hợp thêm mô hình dự báo để tránh việc hệ thống xóa nhầm các nội dung có tiềm năng bùng nổ lưu lượng truy cập.

Để giải quyết bài toán trên, hệ thống được thiết kế theo kiến trúc phân tán đa thành phần để xử lý dữ liệu luồng và theo lô. Cụ thể, hệ thống sử dụng Apache Kafka làm trung gian tiếp nhận và luân chuyển log truy cập tốc độ cao. Apache Spark đóng vai trò xử lý dữ liệu (ETL), gom nhóm theo cửa sổ 15 phút để trích xuất đặc trưng và lưu trữ vào hệ thống Delta Lake. Dựa trên các đặc trưng này, thuật toán học máy XGBoost sẽ dự đoán đối tượng nào sắp trở nên phổ biến (viral), sau đó ghi kết quả vào cơ sở dữ liệu Redis để thuật toán Ripple tra cứu nhanh và đưa ra quyết định giữ lại các đối tượng này trong quá trình khử trùng lặp.

Kết quả thử nghiệm trên bộ dữ liệu Twitter Memcached Traces cho thấy việc tích hợp mô hình Machine Learning mang lại nhiều cải thiện rõ rệt so với hệ thống Ripple nguyên bản. Tỷ lệ Edge Cache Hit Ratio tăng khoảng 8.89%, số lượng yêu cầu được xử lý cục bộ tăng mạnh, trong khi số đối tượng bị xóa khỏi bộ nhớ (Evicted Files) lại giảm đáng kể. Điểm hiệu quả xóa trùng lặp (Dedup Efficiency Score) cũng tăng 7.16%. Mặc dù vậy, lượng yêu cầu truy xuất về máy chủ gốc (Cloud Request) gần như không giảm. Nguyên nhân là do mô hình hiện tại mới chỉ can thiệp thụ động nhằm bảo vệ các đối tượng đang có sẵn trong bộ nhớ đệm, chứ chưa thể chủ động kéo các tệp dữ liệu mới về.

Từ kết quả trên, nhóm nghiên cứu cũng chỉ ra một số hạn chế như độ trễ nhận diện nội dung viral còn cao (do cửa sổ tính toán kéo dài 15 phút) và giới hạn phần cứng khiến mô hình chưa được huấn luyện với toàn bộ dữ liệu. Trong tương lai, định hướng phát triển quan trọng nhất là bổ sung cơ chế phân phối (prefetching) và nhân

bản (replicate) thông minh để chủ động đưa nội dung có giá trị về các cụm máy chủ biên. Ngoài ra, việc ứng dụng các kiến trúc học sâu chuyên biệt cho dữ liệu chuỗi thời gian như LSTM, TCN hoặc mô hình đồ thị (GNN) cũng sẽ được xem xét để nâng cao hiệu quả dự báo.

Chương 1. BỘ DỮ LIỆU VÀ XỬ LÝ ĐẶC TRƯNG

1.1. Bối cảnh nghiên cứu

Điện toán biên đang trở thành một hướng triển khai quan trọng cho các ứng dụng có yêu cầu cao về độ trễ. Thay vì luôn truy xuất dữ liệu từ trung tâm dữ liệu hoặc cloud, hệ thống có thể đặt dữ liệu gần người dùng hơn tại các máy chủ biên. Cách tiếp cận này giúp giảm thời gian phản hồi, giảm áp lực lên mạng lõi và tăng chất lượng trải nghiệm người dùng. Trong các hệ thống phân phối nội dung, nhiều object có thể được cache đồng thời trên nhiều edge node để phục vụ các khu vực địa lý khác nhau. Việc nhân bản dữ liệu giúp tăng khả năng phục vụ tại Edge, nhưng cũng tạo ra hiện tượng dư thừa dữ liệu. Khi dung lượng cache tại Edge bị giới hạn, dữ liệu dư thừa làm giảm hiệu quả sử dụng bộ nhớ và có thể khiến các object quan trọng bị loại bỏ sớm. Bài toán đặt ra là làm thế nào để vừa giảm trùng lặp dữ liệu tại Edge, vừa không làm giảm khả năng phục vụ nhanh của hệ thống. Đây là bối cảnh mà bài báo Ripple được lựa chọn làm nền tảng nghiên cứu cho đề tài.

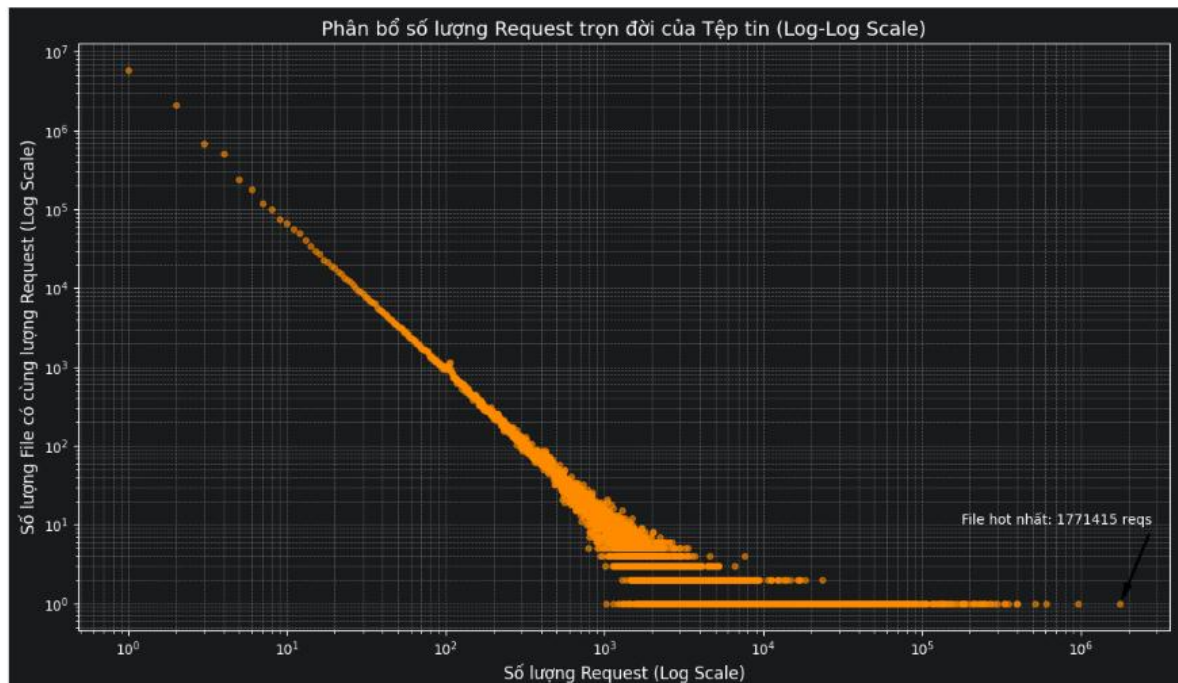
1.2. Bài báo nền tảng: Ripple

Bài báo “Ripple: Enabling Decentralized Data Deduplication at the Edge” được công bố trên IEEE Transactions on Parallel and Distributed Systems, tập 36, số 1, tháng 01/2025. Bài báo đề xuất Ripple như một cơ chế xóa trùng lặp dữ liệu phi tập trung dành cho môi trường điện toán biên. Thay vì sử dụng một bộ điều phối tập trung để nắm toàn bộ trạng thái dữ liệu, Ripple cho phép mỗi máy chủ biên duy trì một chỉ mục cục bộ gọi là cây Ripple. Chỉ mục này lưu thông tin về dữ liệu đang được lưu trên các máy chủ lân cận. Nhờ đó, mỗi máy chủ biên có thể tự phát hiện đối tượng bị trùng lặp, lựa chọn bản sao cần xóa và vẫn đảm bảo rằng dữ liệu còn tồn tại ở vị trí có thể truy xuất trong giới hạn độ trễ cho phép. Ripple có ba mục tiêu chính: nhận diện dữ liệu trùng lặp, xóa dữ liệu dư thừa mà không vi phạm ràng buộc độ trễ, và đảm bảo tính khả dụng của dữ liệu sau khi xóa trùng lặp.

Ngoài ra, cấu trúc chỉ mục có thể mở rộng hoặc thu hẹp tùy theo biến động lưu lượng và trạng thái lưu trữ tại biên.[1]

1.3. Lý do chọn đề tài

Ripple giải quyết tốt bài toán dung lượng cache, nhưng cơ chế vận hành vẫn mang tính phản ứng thụ động. Xóa trùng lặp thường được kích hoạt khi máy chủ biên đạt ngưỡng đầy bộ nhớ đệm. Tại thời điểm đó, hệ thống dựa vào trạng thái hiện tại của dữ liệu và các ràng buộc về độ trễ để quyết định xóa bản sao nào.



Hình 1. Phân bố số lượng Request

Điểm hạn chế khác nữa là Ripple chưa có nhận thức về giá trị tương lai của nội dung. Trong thực tế, phân phối các yêu cầu thường có độ lệch cao: một số ít đối tượng nhận phần lớn lượt truy cập, trong khi phần lớn đối tượng còn lại thuộc nhóm long-tail (số lượng truy cập rất thấp). Nếu một đối tượng bất ngờ bùng nổ lượng truy cập nhưng đã bị Ripple loại bỏ trước đó, hệ thống sẽ gặp tình trạng cache miss, buộc các yêu cầu tiếp theo phải truy xuất từ máy chủ gốc (cloud) hoặc nhảy qua

nhiều node hơn, làm tăng độ trễ đáng kể. Vì vậy, nhóm đặt câu hỏi nghiên cứu: nếu bổ sung một mô hình dự báo độ phổ biến nội dung vào Ripple, hệ thống có thể tránh xóa nhầm các đối tượng có lưu lượng truy cập cao và cải thiện hiệu quả phục vụ tại máy chủ biên hay không?

1.4. Mục tiêu của đề tài

- Xây dựng luồng dữ liệu thời gian thực để thu thập yêu cầu truy cập từ hệ thống Edge/Ripple.
- Sử dụng Spark để xử lý luồng yêu cầu, gom nhóm theo cửa sổ thời gian và tạo đặc trưng cho mô hình học máy.
- Huấn luyện mô hình XGBoost để dự báo đối tượng có khả năng phổ biến trong cửa sổ thời gian tiếp theo.
- Lưu kết quả dự báo vào Redis để Ripple tra cứu nhanh trong quá trình xóa trùng lặp.
- So sánh hiệu quả của hệ thống khi không tích hợp và khi có tích hợp mô hình Machine Learning thông qua các độ đo cache hit, cloud request, eviction và dedup efficiency.

1.5. Phạm vi thực hiện

Đề tài tập trung vào mô phỏng và đánh giá bước đầu trên môi trường thực nghiệm. Dữ liệu đầu vào được lấy từ Twitter Memcached Traces - Cluster 52 và được biến đổi để mô phỏng yêu cầu truy cập đối tượng đến các máy chủ biên. Hệ thống hiện tại chưa triển khai đầy đủ cơ chế prefetch hoặc replicate chủ động, mà chủ yếu dùng kết quả dự báo để bảo vệ đối tượng được dự đoán viral khỏi bị xóa khi Ripple thực hiện xóa trùng lặp. Phạm vi đánh giá bao gồm hai cấu hình: hệ thống Ripple không tích hợp mô hình Machine Learning và hệ thống Ripple có tích hợp luồng dự báo độ

phổ biến. Các kết quả được phân tích nhằm xác định mức cải thiện của chất lượng bộ nhớ đệm tại biên và các hạn chế còn tồn tại.

Chương 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Edge caching và data deduplication

Edge caching là kỹ thuật lưu trữ tạm thời nội dung tại các máy chủ gần người dùng cuối. Khi request được phục vụ trực tiếp tại Edge, hệ thống giảm được độ trễ truy xuất và giảm số lần phải lấy dữ liệu từ cloud. Tuy nhiên, vì nhiều edge node có thể lưu cùng một object, hệ thống dễ phát sinh dữ liệu trùng lặp.

Data deduplication là quá trình phát hiện và loại bỏ các bản sao dư thừa. Trong môi trường điện toán biên, xóa trùng lặp không thể chỉ tối ưu dung lượng mà còn phải xét đến độ trễ. Một đối tượng có thể bị xóa khỏi một node nếu vẫn còn bản sao ở máy chủ biên lân cận và việc truy xuất bản sao đó không vượt quá bước nhảy vượt ngưỡng hoặc độ trễ cho phép.

2.2. Apache Kafka

Kafka là nền tảng truyền phát sự kiện phân tán theo mô hình công bố - đăng ký (publish-subscribe). Trong hệ thống của đề tài, Kafka đóng vai trò giao tiếp dữ liệu (message broker) để tiếp nhận yêu cầu truy cập thô từ Ripple và chuyển tiếp dữ liệu đã được Spark xử lý sang AI Worker. Việc dùng Kafka giúp tách rời các thành phần producer, processing và consumer, đồng thời hỗ trợ mở rộng theo partition.

2.3. Apache Spark và Structured Streaming

Apache Spark là hệ thống tính toán phân tán phục vụ xử lý dữ liệu lớn. Spark Structured Streaming được sử dụng để đọc dữ liệu từ Kafka, parse JSON log, lọc các yêu cầu phù hợp, gom nhóm theo cửa sổ 15 phút và tạo các đặc trưng đầu vào cho mô hình học máy. Spark cũng ghi dữ liệu trung gian vào tầng lưu trữ Silver để phục vụ batch labeling và retraining.

2.4. Redis

Redis là cơ sở dữ liệu key-value lưu dữ liệu chủ yếu trong bộ nhớ RAM, phù hợp cho các truy vấn có độ trễ thấp. Trong hệ thống, Redis lưu các object được AI Worker dự đoán là viral với TTL. Khi Ripple chuẩn bị xóa một danh sách các ứng viên, hệ thống truy vấn Redis để xác định đối tượng nào cần được bảo vệ.

2.5. Delta Lake

Delta Lake được sử dụng như lớp lưu trữ lakehouse cho các tầng dữ liệu Bronze, Silver và Gold. Bronze lưu log thô, Silver lưu feature đã xử lý, còn Gold lưu dữ liệu đã gán nhãn để huấn luyện lại mô hình. Cách tổ chức này giúp pipeline có khả năng tái sử dụng dữ liệu và hỗ trợ cập nhật theo batch.

2.6. XGBoost

XGBoost là thuật toán gradient boosting trên cây quyết định, thường được sử dụng cho các bài toán phân loại có dữ liệu dạng bảng. Trong đề tài, XGBoost được dùng để dự báo nhãn `is_viral_next_window` dựa trên các đặc trưng như số request hiện tại, số request trước đó, số edge node hoạt động, tốc độ tăng trưởng và chu kỳ thời gian trong ngày.

2.7. Tổng hợp vai trò của các công nghệ

Bảng 1. Tổng hợp vai trò của các công nghệ

Công nghệ	Vai trò
Apache Spark	Xử lý dữ liệu thành các đặc trưng đầu vào cho mô hình.
Apache Kafka	Đóng vai trò là Message Broker chịu trách nhiệm tiếp nhận, lưu trữ tạm thời và luân chuyển luồng dữ liệu lớn tốc độ cao theo

	thời gian thực đến các module xử lý tiếp theo.
Redis	Làm bộ nhớ đệm (Cache) lưu trữ kết quả của mô hình ML và cho phép các node truy cập nhanh.
Delta Lake	Làm kho lưu trữ kết quả xử lý của Spark.
XGBoost	Là mô hình Máy học chính để đưa ra dự đoán dựa trên các đặc trưng đầu vào từ Spark.

Chương 3. BỘ DỮ LIỆU VÀ XỬ LÝ ĐẶC TRƯNG

3.1. Tổng quan về bộ dữ liệu

Tên bộ dữ liệu: Twitter Memcached Traces - Cluster 52.

Đây là bộ dữ liệu lịch sử truy cập bộ nhớ đệm (cache trace) quy mô lớn, nổi tiếng được Twitter công bố, ghi lại chi tiết các luồng yêu cầu đến cụm máy chủ bộ nhớ đệm nội bộ. Bộ dữ liệu cung cấp góc nhìn về hành vi lưu lượng truy cập ở cấp độ production của một trong những mạng xã hội có lượng xử lý thời gian thực lớn nhất thế giới.

Đặc điểm của Cluster 52 là lưu lượng truy cập có tính bất đối xứng cao. Nó phản ánh rõ rệt hiện tượng một số ít nội dung (như các đoạn text viral) được truy xuất với tần suất cực cao, trong khi phần lớn dữ liệu khác nằm ở dải tần suất thấp (long-tail distribution). Đồng thời bộ dữ liệu này cũng được khuyến nghị để thử nghiệm độ hiệu quả của các thuật toán tiếp nhận (admission) và chính sách loại bỏ (eviction) dữ liệu ra khỏi bộ nhớ.[2]

3.2. Xử lý đặc trưng

Bộ dữ liệu ban đầu cung cấp các đặc trưng gốc như sau:

- timestamp: thời gian nhận request (sec)
- anonymized key: khoá gốc đã được mã hoá
- key size: kích thước key
- value size: kích thước value
- client id: mã ẩn danh của client request
- operation: phương thức
- TTL: thời gian sống

Từ các đặc trưng này, nhóm sẽ biến đổi như sau để tạo ra các đặc trưng phù hợp cho quá trình giả lập:

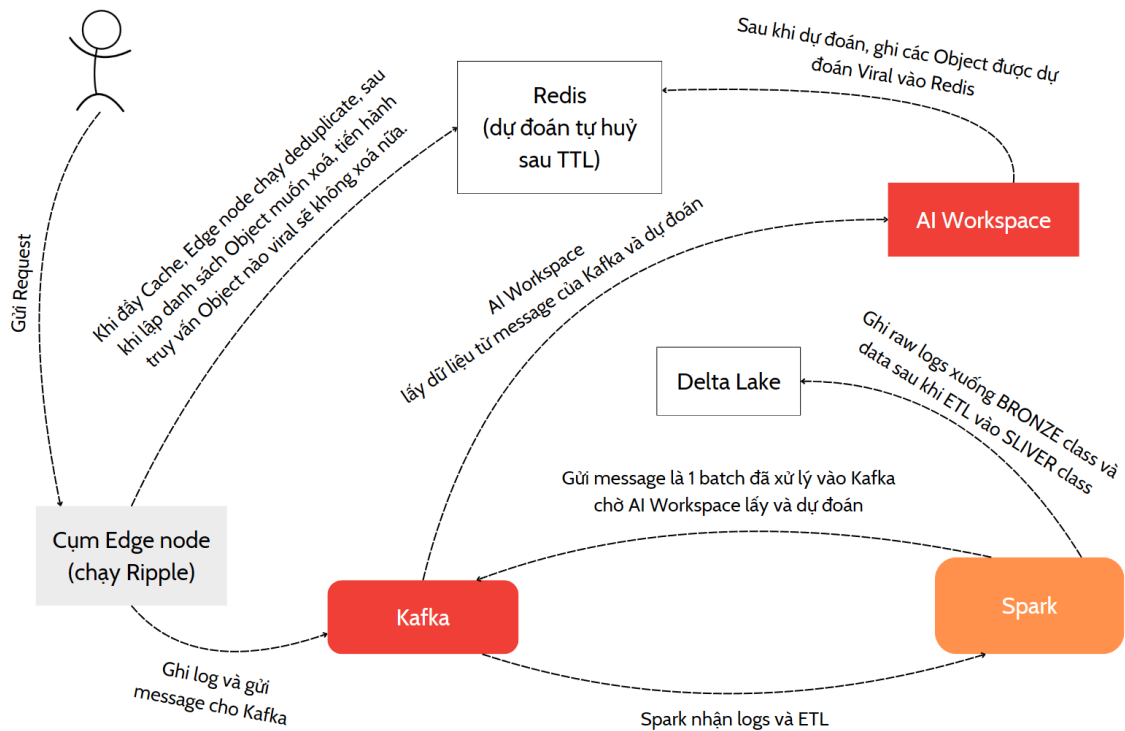
- timestamp: giữ nguyên từ bộ dữ liệu.
- key: dùng anonymized key để hash thành một key mới lưu dạng Long để phù hợp với mã nguồn Ripple.
- node_id: dùng client_id chia lấy dư cho 30 để tạo đặc trưng cho biết node nào nhận được request đó.
- operation: giữ nguyên
- Loại bỏ các đặc trưng không dùng đến: key size, value size, TTL

Chương 4. THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

4.1. Kiến trúc tổng thể hệ thống

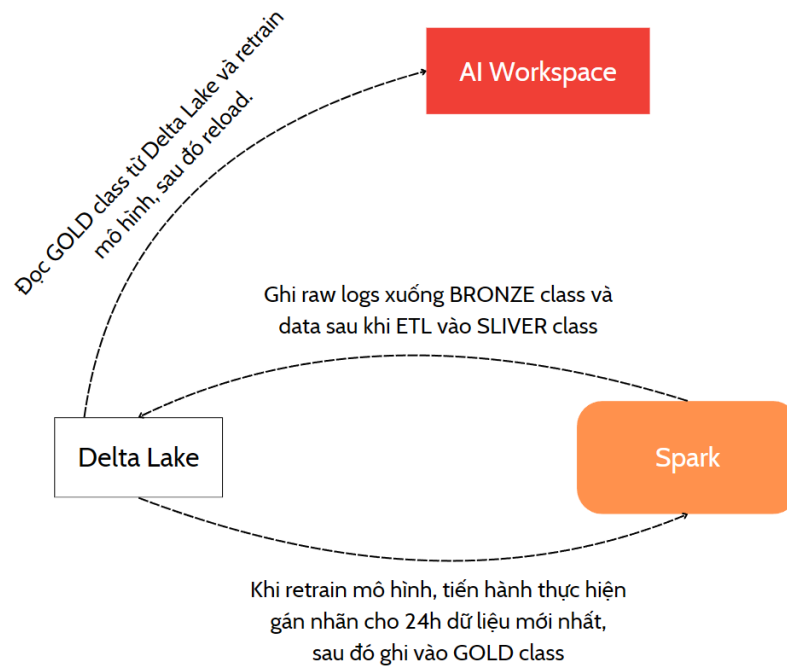
Hệ thống được thiết kế theo kiến trúc phân tán đa thành phần, bao gồm: Ripple đảm nhiệm thuật toán khử trùng lặp tại biên (edge); Kafka làm trung gian điều phối luồng dữ liệu; Spark chịu trách nhiệm xử lý ETL theo thời gian thực; AI Worker vận hành mô hình dự đoán XGBoost; và Redis đóng vai trò lưu trữ kết quả dự báo để Ripple tra cứu.

Luồng xử lý thời gian thực diễn ra như sau: Khởi điểm, các Edge Node tiếp nhận, phục vụ yêu cầu (request) từ người dùng và đồng thời ghi nhận nhật ký truy cập (log). Lượng log này được đẩy liên tục vào Kafka thông qua topic `edge_node_req_logs`. Tại đây, Spark hoạt động dưới vai trò consumer, tiến hành xử lý dữ liệu và trích xuất các đặc trưng (features) đầu vào cho mô hình học máy. Các features này sau đó được lưu trữ đồng thời vào phân lớp SILVER của Delta Lake và đẩy sang topic `cdn_features_stream`. Tiếp theo, phân hệ AI sẽ nhận dữ liệu từ luồng này, tiến hành dự báo khả năng viral của object và ghi các `object_id` đạt điều kiện vào Redis. Cuối cùng, khi dung lượng cache tại các Edge Node đạt giới hạn, thuật toán Ripple sẽ kích hoạt quá trình khử trùng lặp. Hệ thống tiến hành lập danh sách các object dự kiến giải phóng, sau đó đối chiếu với Redis để giữ lại (loại khỏi danh sách xóa) những object đã được dự báo là viral.



Hình 2. Sơ đồ mô hình hệ thống tổng thể

Song song với luồng xử lý thời gian thực, hệ thống vận hành thêm một luồng xử lý theo lô (batch processing) định kỳ. Tiến trình này đảm nhiệm vai trò gán nhãn tự động cho các dữ liệu tại phân lớp SILVER, sau đó chuyển đổi và lưu trữ sang phân lớp GOLD. Tập dữ liệu chuẩn hóa tại lớp GOLD này sẽ được sử dụng làm tài nguyên đầu vào phục vụ cho việc tái huấn luyện (retrain) mô hình dự đoán.



Hình 3. Sơ đồ luồng Batch

4.2. Mở rộng mã nguồn Ripple để giả lập môi trường phân tán

Trong phạm vi đề tài, nhóm sử dụng mã nguồn demo của Ripple làm nền tảng để giả lập hệ thống lưu trữ phân tán tại biên. Mã nguồn Ripple ban đầu chủ yếu phục vụ mục đích minh họa thuật toán xóa trùng lặp (deduplication) trên quy mô nhỏ, với lượng dữ liệu giả lập chỉ ở mức vài trăm dòng và chưa có cơ chế tiếp nhận luồng dữ liệu truy cập theo thời gian thực.

Để phù hợp với mục tiêu xây dựng hệ thống xử lý dữ liệu lớn, nhóm đã tham khảo thêm mã nguồn MagNet và tiến hành mở rộng luồng xử lý dữ liệu của Ripple. Cụ thể, nhóm xây dựng cơ chế đọc dữ liệu truy cập từ tập log Memcached Traces, chuyển đổi mỗi dòng log thành một yêu cầu mô phỏng, ánh xạ yêu cầu đến các máy chủ biên tương ứng và đưa dữ liệu vào hệ thống Ripple theo thứ tự thời gian. Hệ thống giả lập gồm 30 máy chủ biên, trong đó mỗi yêu cầu được định tuyến đến một máy chủ biên dựa trên thông tin client hoặc node id đã được ánh xạ từ dữ liệu gốc.

Việc mở rộng này giúp Ripple không còn chỉ chạy trên dữ liệu mẫu nhỏ, mà có thể tiếp nhận và xử lý luồng yêu cầu truy cập lớn theo hướng gần realtime. Đồng thời, mỗi yêu cầu cũng được gửi song song sang Kafka để phục vụ luồng xử lý dữ liệu thời gian thực. Nhờ đó, hệ thống có thể mô phỏng đồng thời hai quá trình: quá trình xóa trùng lặp(deduplication) diễn ra tại các máy chủ biên và quá trình thu thập log, trích xuất đặc trưng, dự đoán độ phổ biến nội dung bằng mô hình Machine Learning.[3]

4.3. Cấu hình Kafka topic

Topic `edge_node_req_logs`: Đây là topic hứng dữ liệu request thô ban đầu.

- `num_partitions = 3`
- `replication_factor = 1`
- `retention.ms = 86400000` (tương đương 1 ngày),
- `cleanup.policy = delete`.

Topic `cdn_features_stream`: Đây là topic dùng để Spark đẩy các luồng đặc trưng (features) đã được tính toán sang cho AI Worker dự đoán.

- `num_partitions = 3`
- `replication_factor = 1`.
- `retention.ms = 3600000` (1 giờ)
- `Cleanup.policy = delete`.

4.4. Luồng Realtime

1. Spark đọc message JSON từ topic `edge_node_req_logs`.
2. Tiến hành lọc operation = get và gom nhóm theo `object_id` và các cửa sổ 15 phút không trùng nhau.
3. Tính toán feature:

- req_count_current: số lượng request trong cửa sổ hiện tại.
 - active_edges_count: số node cùng nhận được request về object này trong cửa sổ hiện tại.
 - req_count_previous: số lượng request trong cửa sổ trước đó.
 - growth_rate: tỉ lệ tăng trưởng
 - hour_of_day, hour_sin: đặc trưng về thời gian.
4. Dữ liệu feature được lưu vào Silver và publish sang topic `cdn_features_stream`.
 5. AI Worker nhận được dữ liệu đã xử lý và tiến hành dự báo object viral sau đó ghi kết quả vào Redis.

4.5. Luồng Batch

1. Khi đến thời gian đã thiết lập định kỳ, Spark đọc 24h dữ liệu mới nhất từ lớp SILVER của Delta Lake
2. Tiến hành gán nhãn cho từng cửa sổ:
 - Gán nhãn hiện tại: Sử dụng P75 trên tổng thể để làm mức sàn loại bỏ nhiễu, sau đó tính P95 làm ngưỡng cho riêng từng window. Các đối tượng thỏa mãn cả 2 mức trên sẽ được dán nhãn là viral tại thời điểm hiện tại (`is_viral_now`).
 - Dịch chuyển nhãn (Label Shifting): Lấy nhãn của cửa sổ thời gian 15 phút tiếp theo (`is_viral_now`) và gán thành nhãn mục tiêu (`is_viral_next_window`) cho tập đặc trưng của cửa sổ hiện tại.
3. Dùng dữ liệu ở lớp GOLD để tiến hành retrain lại mô hình.

4.6. Tích hợp với Ripple

Trong Ripple nguyên bản, khi một máy chủ biên đạt ngưỡng giới hạn lưu trữ tối đa (`storageUpperLimit`), sẽ tiến hành xóa trùng lặp (`deduplicate`) để xóa các bản sao dư

thừa. Trong phiên bản tích hợp mô hình Machine Learning, trước khi xóa, Ripple kiểm tra Redis để xác định đối tượng nào đang được dự báo là viral. Những đối tượng này được giữ lại trong bộ nhớ đệm, còn các đối tượng không phổ biến (viral) tiếp tục được xử lý theo chiến lược xóa trùng lặp (deduplication). Cơ chế này biến kết quả dự báo thành một tham số kiểm soát của Ripple. Ripple không chỉ xét vị trí bản sao và ràng buộc độ trễ, mà còn xét giá trị truy cập dự báo của object. Đây là điểm mở rộng quan trọng giúp Ripple chuyển từ “nhận thức về độ trễ” sang “nhận thức về giá trị nội dung”.

Chương 5. HUẤN LUYỆN MÔ HÌNH BAN ĐẦU VÀ THỰC NGHIỆM

5.1. Thiết lập huấn luyện

Thuật toán XGBoost được lựa chọn để xây dựng mô hình dự đoán chính. Ban đầu nhóm dự định huấn luyện trên 24h dữ liệu thực tế (tương đương khoảng 12 part). Tuy nhiên do giới hạn về cấu hình của thiết bị nên chỉ có thể huấn luyện trên 1 part (tương đương 2,26h dữ liệu thực tế).[4]

Mô hình được huấn luyện trên Part001 chứa khoảng 15 triệu dòng dữ liệu. Bộ siêu tham số tối ưu tìm bằng GridSearchCV như sau:

- `n_estimators=200`
- `max_depth=5`
- `learning_rate=0.1`
- `eval_metric='logloss'`
- `random_state=42,`
- `scale_pos_weight=viral_ratio`

Việc đối chiếu hiệu suất dự đoán trên Part001 và Part003 cho thấy mô hình không bị sụt giảm mạnh về độ chính xác khi áp dụng trên một phân vùng dữ liệu mới. Vì thế vẫn có thể áp dụng vào bước đầu của hệ thống.

```
--- Báo cáo phân loại (Classification Report) ---
              precision    recall  f1-score   support

    0           0.98         0.99         0.99     3258446
    1           0.78         0.63         0.70     162738

 accuracy                   0.97     3421184
 macro avg           0.88         0.81         0.84     3421184
weighted avg           0.97         0.97         0.97     3421184
```

Hình 4. Báo cáo phân loại của mô hình trên Part001

```

--- Báo cáo phân loại (Classification Report) ---
              precision    recall  f1-score   support

         0              0.98         0.99         0.99    14985813
         1              0.79         0.55         0.64     715556

 accuracy              0.97    15701369
  macro avg              0.88         0.77         0.82    15701369
 weighted avg              0.97         0.97         0.97    15701369

```

Hình 5. Báo cáo phân loại của mô hình trên Part003

5.2. Đánh giá mô hình máy học

Trên tập dữ liệu Part001, mô hình đạt độ chính xác (Accuracy) khá cao ở mức 0.97. Tuy nhiên, do đặc thù dữ liệu mất cân bằng nghiêm trọng (tỷ lệ khoảng 20:1), chỉ số này không phản ánh bức tranh hiệu suất thực tế. Do đó, nhóm đã tập trung đánh giá thêm qua các chỉ số Precision, Recall và F1-score. Ở mô hình này, ngưỡng quyết định (Threshold) được chủ động thiết lập ở mức rất cao (0.95), buộc mô hình phải đạt độ tự tin gần như tuyệt đối mới kết luận một tệp là "viral". Điều này lý giải việc Precision đạt mức khả quan (khoảng 0.8) trong khi Recall chỉ dừng ở 0.63.

Lựa chọn này được thiết kế để phục vụ trực tiếp cho bài toán tối ưu hệ thống: Khi cơ chế Ripple được kích hoạt, đồng nghĩa với việc node đó đã chạm ngưỡng giới hạn về cache. Nếu mô hình đưa ra quá nhiều dự đoán sai (False Positive - nhận nhầm tệp bình thường thành viral), không gian cache sẽ bị lãng phí, khiến quá trình khử trùng lặp trở nên kém hiệu quả. Do vậy, nhóm chấp nhận đánh đổi: thà bỏ lỡ một vài tệp viral thực sự (False Negative), còn hơn lưu trữ nhầm, qua đó đảm bảo tài nguyên hệ thống chỉ tập trung xử lý và tối ưu cho những tệp có xác suất viral cao nhất.

```

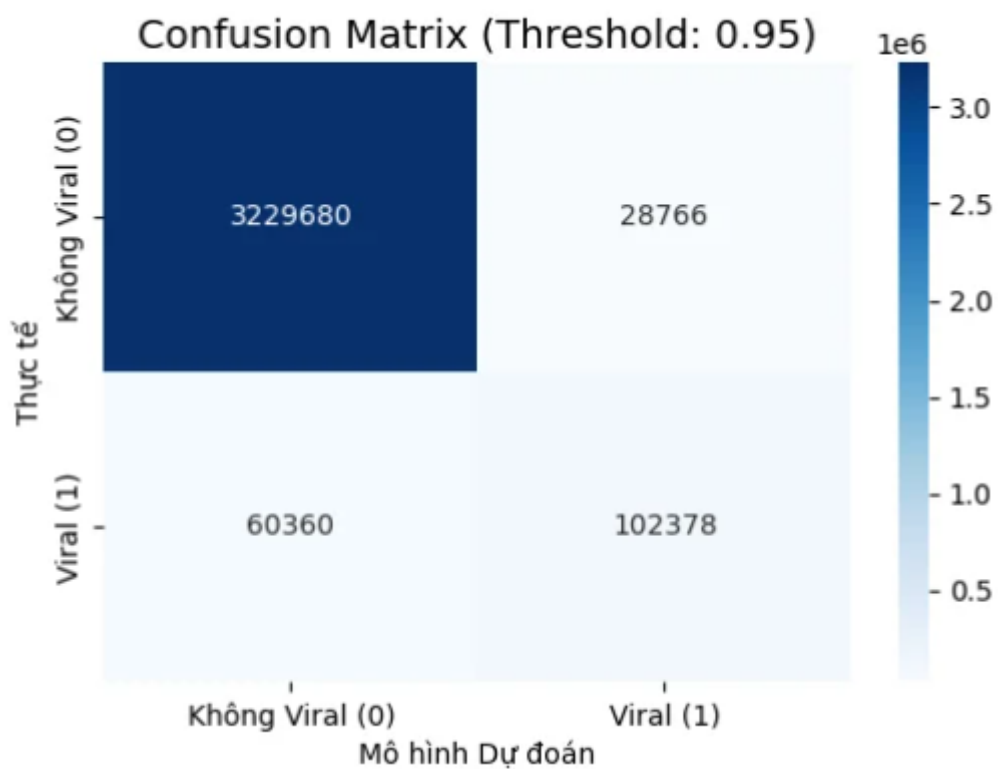
--- Báo cáo phân loại (Classification Report) ---
              precision    recall  f1-score   support

      0:       0.98         0.99         0.99     3258446
      1:       0.78         0.63         0.70     162738

 accuracy:      0.97     3421184
 macro avg:      0.88     3421184
 weighted avg:      0.97     3421184

```

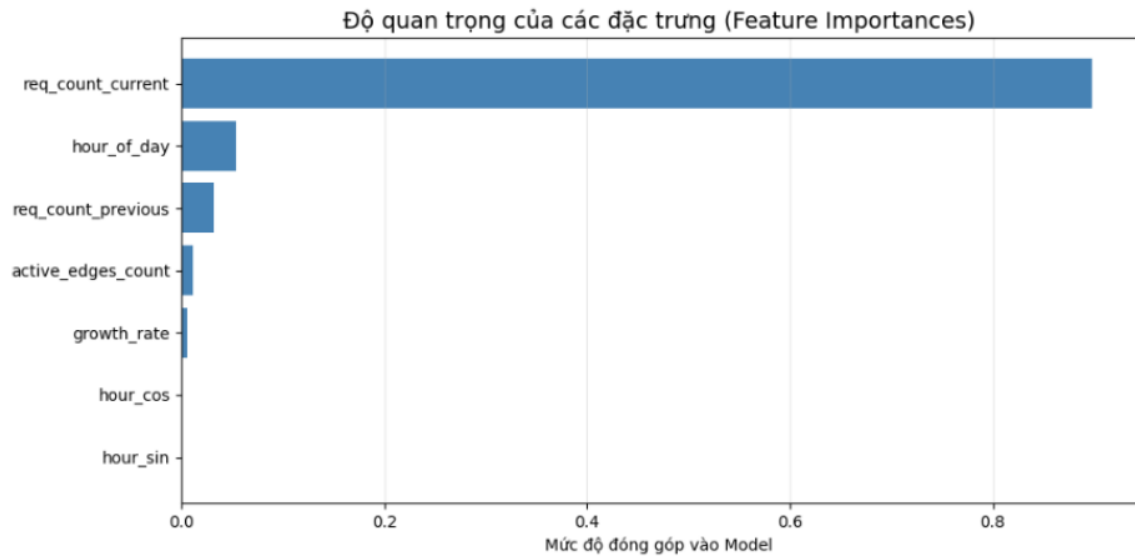
Hình 6. Báo cáo phân loại của mô hình trên Part001



Hình 7. Ma trận nhầm lẫn của mô hình trên Part001 với Threshold 0.95

Tuy nhiên, mô hình hiện tại đang phụ thuộc lớn vào đặc trưng `req_count_current`, làm giảm mức độ đóng góp (feature importance) của các đặc trưng khác như `growth_rate`.

Trong các bước phát triển tiếp theo, nhóm có thể tiến hành thử nghiệm loại bỏ đặc trưng `req_count_current` nhằm đánh giá khả năng mô hình trích xuất các mẫu (patterns) ẩn từ những đặc trưng còn lại.



Hình 8. Biểu đồ Độ quan trọng của các đặc trưng

5.3. Cấu hình vận hành thí nghiệm

- Tổng số dòng log mô phỏng: 420,000.
- Tốc độ gửi (số lượng yêu cầu đưa vào): 8,000 req/s.
- Bộ dữ liệu dùng để giả lập: Cluster-part002.
- Số bước nhảy(hop-num): 5.
- DedupBatchSize(Số lượng yêu cầu được gửi trong 1 batch): 10,000.
- Kafka topic `edge_node_req_logs`: 3 partitions.
- Kafka topic `cdn_features_stream`: 3 partitions.

5.4. Các độ đo đánh giá hệ thống

Bảng 2. Các độ đo đánh giá

Độ đo	Ý nghĩa	Công thức/Ghi chú
Edge Cache Hit Ratio	Tỷ lệ request được xử lý tại Edge	$(\text{Local Hit} + \text{Neighbor Hit}) / \text{Total Requests}$
Local Hit Requests	Số request được xử lý trực tiếp tại edge node đang nhận request	Giá trị càng cao càng tốt
Cloud Request	Số request phải lấy từ cloud/server gốc	Giá trị càng thấp càng tốt
Evicted Files	Số object bị xóa khỏi cache	Phản ánh áp lực eviction
Dedup Efficiency Score	Hiệu quả phục vụ tại Edge trên mỗi đơn vị cache được sử dụng	$\text{Edge Cache Hit Ratio} / \text{Avg Storage Utilization}$

5.5. Kết quả thực nghiệm và phân tích kết quả

Bảng 3. Bảng kết quả so sánh trước và sau khi tích hợp ML

Độ đo	Tích hợp mô hình	Không tích hợp mô hình	Nhận xét
Edge Cache Hit Ratio	0.3603	0.3309	Tăng khoảng 8.89% tương đối
Local Hit Requests	100,018	75,981	Tăng 24,037 request
Cloud Request	133,360	133,393	Gần như không đổi

Evicted Files	184,784	210,879	Giảm 26,095 object
Dedup Efficiency Score	0.5835	0.5445	Tăng khoảng 7.16% tương đối

Kết quả cho thấy việc tích hợp mô hình dự đoán đã cải thiện chất lượng bộ nhớ đệm (cache) tại máy chủ biên. Edge Cache Hit Ratio tăng từ 0.3309 lên 0.3603, tương đương tăng khoảng 8.89% theo tỷ lệ tương đối. Local Hit Requests tăng đáng kể từ 75,981 lên 100,018, cho thấy nhiều request hơn được phục vụ trực tiếp tại máy chủ biên nhận yêu cầu truy cập thay vì phải truy xuất xa hơn. Số lượng Evicted Files giảm từ 210,879 xuống 184,784. Điều này phản ánh cơ chế bảo vệ object viral giúp hạn chế xóa các nội dung có khả năng tiếp tục được truy cập. Dedup Efficiency Score cũng tăng từ 0.5445 lên 0.5835, cho thấy hệ thống có tích hợp mô hình dự đoán sử dụng bộ nhớ đệm hiệu quả hơn xét trên cùng mức độ sử dụng lưu trữ trung bình. Tuy nhiên, Cloud Request gần như không thay đổi, chỉ giảm từ 133,393 xuống 133,360. Nguyên nhân chính là do hệ thống hiện tại chỉ mới can thiệp thụ động tại bước khử trùng lặp (giữ lại các đối tượng dự báo viral đang có sẵn trong cache). Khả năng chủ động phân phối (prefetch) hoặc nhân bản (replicate) các nội dung này về các edge node phù hợp trước khi lưu lượng bùng nổ vẫn chưa được triển khai. Vì vậy, mô hình đã cải thiện khả năng giữ nội dung có giá trị trong bộ nhớ đệm, nhưng chưa đủ để giảm mạnh cloud backhaul tổng thể. Từ kết quả trên có thể kết luận rằng tích hợp AI vào Ripple là hướng đi khả thi. Phiên bản hiện tại đã chứng minh tác động tích cực lên hit ratio, local hit và eviction. Để đạt tác động rõ hơn lên Cloud Request, cần bổ sung tầng điều phối chủ động dựa trên dự báo viral, chẳng hạn prefetch về máy chủ biên gần cụm người dùng hoặc replicate sang các máy chủ biên có xác suất nhận các yêu cầu truy cập cao.

Chương 6. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

6.1. Hạn chế hiện tại

- Độ trễ nhận diện viral còn cao do Spark sử dụng tumbling window 15 phút. Hệ thống cần ít nhất một window để tích lũy dữ liệu trước khi tạo feature đủ ý nghĩa cho dự báo.
- Mô hình được huấn luyện trên tập dữ liệu nhỏ hơn dự kiến do hạn chế phần cứng. Khi triển khai trên dữ liệu thực tế lớn hơn hoặc thay đổi phân phối, độ chính xác có thể giảm.
- Hệ thống mới bảo vệ đối tượng đã nằm trong bộ nhớ đệm, chưa triển khai prefetch hoặc replicate chủ động. Vì vậy Cloud Request tổng thể chưa giảm đáng kể.
- Kết quả thực nghiệm mới dựa trên môi trường mô phỏng, chưa đánh giá đầy đủ các yếu tố vận hành production như lỗi node, độ trễ mạng thật, phân phối địa lý người dùng và chi phí đồng bộ nhiều cụm biên.
- Việc lựa chọn threshold dự báo phổ biến (viral) ảnh hưởng trực tiếp đến trade-off giữa giữ thêm dữ liệu trong bộ nhớ đệm và nguy cơ xóa nhầm đối tượng phổ biến. Phiên bản hiện tại chưa có cơ chế tự điều chỉnh threshold theo tải hệ thống.

6.2. Hướng phát triển

Hướng phát triển quan trọng nhất là bổ sung cơ chế prefetching thông minh. Khi mô hình dự báo một đối tượng có xác suất phổ biến (viral) cao, hệ thống có thể chủ động kéo đối tượng đó về các máy chủ biên gần nhóm người dùng có khả năng phát sinh lượng yêu cầu truy cập cao. Cách này có thể giảm Cloud Request và giảm độ trễ trước khi luồng truy cập bùng nổ. Một hướng khác là triển khai replicate chủ động theo cụm địa lý. Thay vì chỉ lưu đối tượng tại một máy chủ biên, hệ thống có thể dùng dự báo `active_edges_count` và lịch sử truy cập để quyết định số lượng bản

sao cần duy trì. Khi đối tượng hết phổ biến (viral) hoặc TTL hết hạn, hệ thống có thể co lại số bản sao để tiết kiệm bộ nhớ đệm. Về mô hình, có thể thử các kiến trúc học sâu cho chuỗi thời gian như LSTM, TCN hoặc các mô hình graph-based như GNN để học tương quan lan truyền giữa đối tượng và máy chủ biên. Tuy nhiên, cần cân nhắc chi phí tính toán và độ trễ suy luận trong môi trường realtime. Cuối cùng, hệ thống nên bổ sung cơ chế giám sát vận hành gồm dashboard Kafka lag, Spark micro-batch latency, Redis hit/miss và số đối tượng được bảo vệ bởi mô hình Machine Learning. Các chỉ số này giúp đánh giá luồng hệ thống không chỉ về kết quả thuật toán mà còn về khả năng vận hành ổn định.

6.3. Kết luận

Báo cáo đã trình bày quá trình xây dựng hệ thống luồng dữ liệu thời gian thực để dự báo độ phổ biến nội dung tại biên mạng và tích hợp kết quả dự báo vào cơ chế xóa trùng lặp (deduplication) của Ripple. Hệ thống sử dụng Kafka để truyền log và feature stream, Spark để xử lý ETL realtime/batch, Delta Lake để lưu dữ liệu theo các lớp Bronze - Silver - Gold, XGBoost để dự báo đối tượng phổ biến và Redis để cung cấp kết quả dự báo có độ trễ thấp cho Ripple. Kết quả thực nghiệm cho thấy phiên bản có tích hợp mô hình cải thiện Edge Cache Hit Ratio, Local Hit Requests, Evicted Files và Dedup Efficiency Score so với phiên bản không tích hợp. Điều này chứng minh rằng việc bổ sung nhận thức về giá trị nội dung có thể nâng cao chất lượng bộ nhớ đệm tại các máy chủ biên. Tuy nhiên, Cloud Request gần như chưa giảm do hệ thống hiện mới bảo vệ đối tượng trong bộ nhớ đệm, chưa có cơ chế prefetch/replicate chủ động. Trong các phiên bản tiếp theo, nhóm cần tập trung vào prefetching thông minh, replicate theo cụm máy chủ biên, tối ưu độ trễ window và thử nghiệm các mô hình dự báo nâng cao. Những hướng này có thể giúp hệ thống không chỉ tránh xóa nhầm nội dung viral mà còn chủ động đưa nội dung có giá trị đến đúng vị trí trước khi nhu cầu truy cập tăng mạnh.

TÀI LIỆU THAM KHẢO

- [1] R. Luo, Q. He, F. Chen, S. Wu, H. Jin, and Y. Yang, “Ripple: Enabling Decentralized Data Deduplication at the Edge,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 1, pp. 55–66, 2025, doi: 10.1109/TPDS.2024.3493953.
- [2] Juncheng Yang, Yao Yue, and K. V. Rashmi, *A large scale analysis of hundreds of in-memory cache clusters at Twitter*. USENIX Association, 2020.
- [3] J. Peng *et al.*, “MagNet: Cooperative Edge Caching by Automatic Content Congregating,” in *WWW 2022 - Proceedings of the ACM Web Conference 2022*, Association for Computing Machinery, Inc, Apr. 2022, pp. 3280–3288. doi: 10.1145/3485447.3512146.
- [4] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Jun. 2016, doi: 10.1145/2939672.2939785.